

# Nginx.Notes [In-Depth]

Architecture · Config Contexts · Processing

## ⇒ NGINX MASTERCLASS ⇒

### Part 1: Core Architecture & Connections

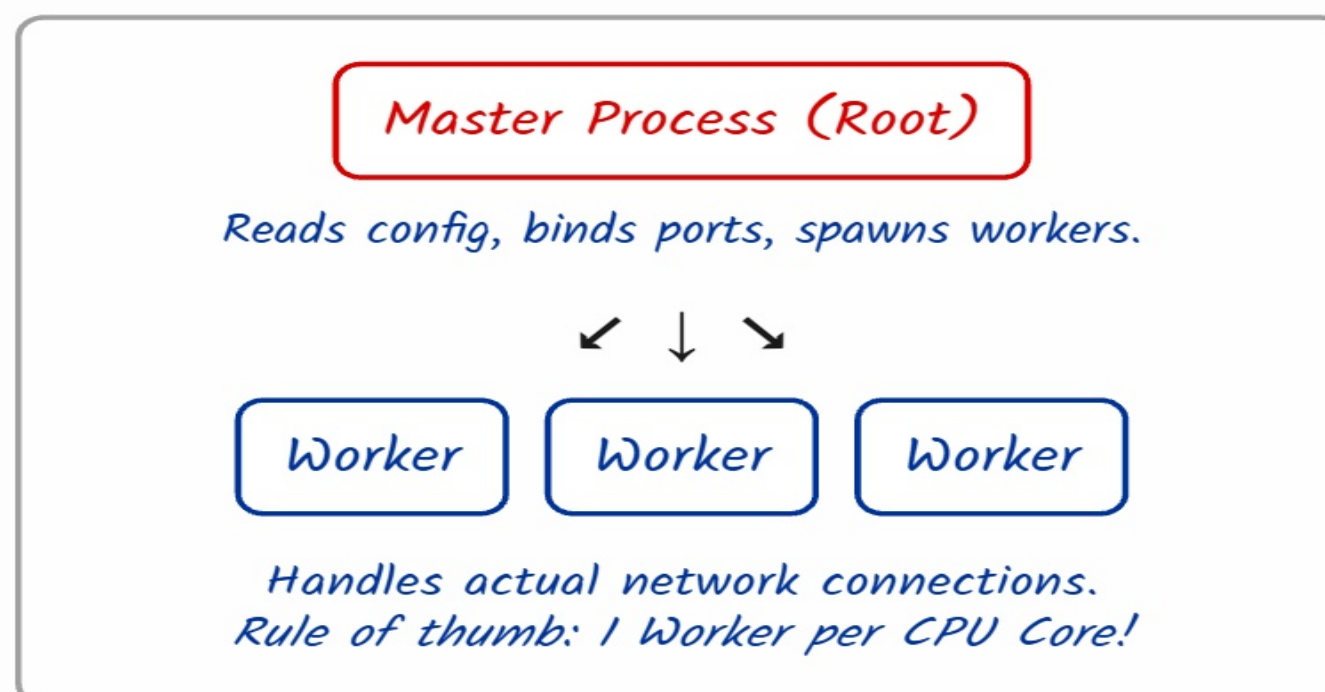
#### ★ 1. The C10K Problem

Traditional servers (like Apache MPM Prefork) spawn a new **Thread or Process** for every connection. High RAM usage + Context Switching = crash under heavy load.

NGINX solves this using an **Event-Driven, Asynchronous, Non-Blocking** architecture.

#### ★ 2. Master vs Worker Processes

NGINX runs one **Master** process and multiple **Worker** processes.



```
# In nginx.conf
worker_processes auto; # Auto detects
cores
worker_cpu_affinity auto; # CPU Pinning
```

#### ★ 3. The Event Loop (Epoll/Kqueue)

Instead of blocking while waiting for a database or disk, the Worker uses an **Event Loop**.

- New connection arrives → Added to Event Queue.
- Worker picks it up → Asks OS for data.
- **Non-blocking:** Worker doesn't wait! It moves to the next event in the queue.
- OS notifies Worker when data is ready.

#### Worker Capacity Formula

$$\text{Max Clients} = \text{worker\_processes} * \text{worker\_connections}$$

```
events {
    worker_connections 1024;
    multi_accept on; # Accept all at once
    use epoll; # Efficient method on Linux
}
```

#### ★ 4. Thread Pools (AIO)

Even non-blocking NGINX can block on **Heavy Disk I/O**.

Solution: Send large file reads to a separate Thread Pool so the Event Loop isn't delayed.

**Crucial Takeaway:** Because workers share the load via event queues, NGINX can handle **tens of thousands** of concurrent connections (HTTP Keep-Alive) with minimal RAM (~2-3MB per 10k connections).

## ★ 5. Config Structure & Contexts

The nginx.conf file is a hierarchical tree of **Contexts** (Blocks) and Directives (Instructions).

### Main context (Global settings)

```
└─ events { ... }
└─ http {
    └─ upstream backend { ... }
    └─ server { // virtual host 1
        └─ location / { ... }
        └─ location /api/ { ... }
    }
    └─ server { // virtual host 2 ... }
}
```

## ★ 6. Context Inheritance

Directives inherit **downwards**. If you declare `root /var/www;` in the server block, all location blocks inherit it unless they override it.

⚠ Arrays (like `add_header`) **do NOT append!** If you define an `add_header` in a location block, it wipes out all headers inherited from the server block!

## ★ 7. Location Matching Logic

How does NGINX choose which location block processes a URI? It doesn't look top-to-bottom. It uses **Strict Priority Modifiers**.

### Matching Priority (Highest to Lowest)

1. **= Exact Match:**  
location = /login (Stops searching immediately)
2. **^~ Preferential Prefix:**  
location ^~ /images/ (If longest prefix has this, skip regex)
3. **~ and ~\* Regex:**  
~ (case-sensitive), ~\* (insensitive).  
Evaluated top-to-bottom.
4. **Standard Prefix:**  
location / (The longest matching prefix wins).

```
# Request: /logo.png
location / {
    # Matches, but NGINX keeps looking
}
location ~* \.(png|jpg|gif)$ {
    # Regex wins! This block handles it.
    expires 30d;
}
```

**Pro-Tip:** Use exact matches (=) for highly trafficked routes (like / or /api) to speed up the processing phase by skipping regex evaluations!

## ★ 8. Root vs Alias (The Classic Trap)

How NGINX translates the URI into a filesystem path.

**ROOT:** Appends the URI to the path.

```
location /app/ {  
    root /var/www/html;  
}  
# URI: /app/index.html  
# Path: /var/www/html/app/index.html
```

**ALIAS:** Replaces the matched location part.

```
location /app/ {  
    alias /var/www/html/;  
}  
# URI: /app/index.html  
# Path: /var/www/html/index.html
```

⚠ Always ensure trailing slashes match on both the location and the alias path!

## ★ 9. Routing: Rewrite vs Return

return →

Fast & Simple HTTP Redirect

rewrite →

Internal URL change via Regex

# RETURN (Best for HTTP → HTTPS)

```
server {  
    listen 80;  
    return 301 https://$host$request_uri;  
}
```

# REWRITE (Internal mapping)

```
rewrite ^/user/(\d+) /profile.php?id=$1 last;  
# 'last' flag stops rewrite phase &  
# searches for new location match.
```

## ★ 10. SPAs & try\_files

For React/Vue/Angular apps, the server must route everything to index.html.

```
location / {  
    try_files $uri $uri/ /index.html;  
}
```

Checks if file exists (\$uri), then if directory exists (\$uri/), if neither, serves index.html internally.

## ★ 11. Proxy Pass Basics

NGINX acts as a middleman, forwarding requests to backend apps (Node.js, Python, PHP-FPM via fastcgi).

### The Trailing Slash Gotcha!

**Request:** /api/users

**No Slash:** proxy\_pass http://backend;  
Sends: /api/users to backend.

**With Slash:** proxy\_pass http://backend/;  
Sends: /users to backend. (Strips the matching part!)

## ★ 12. Passing Headers

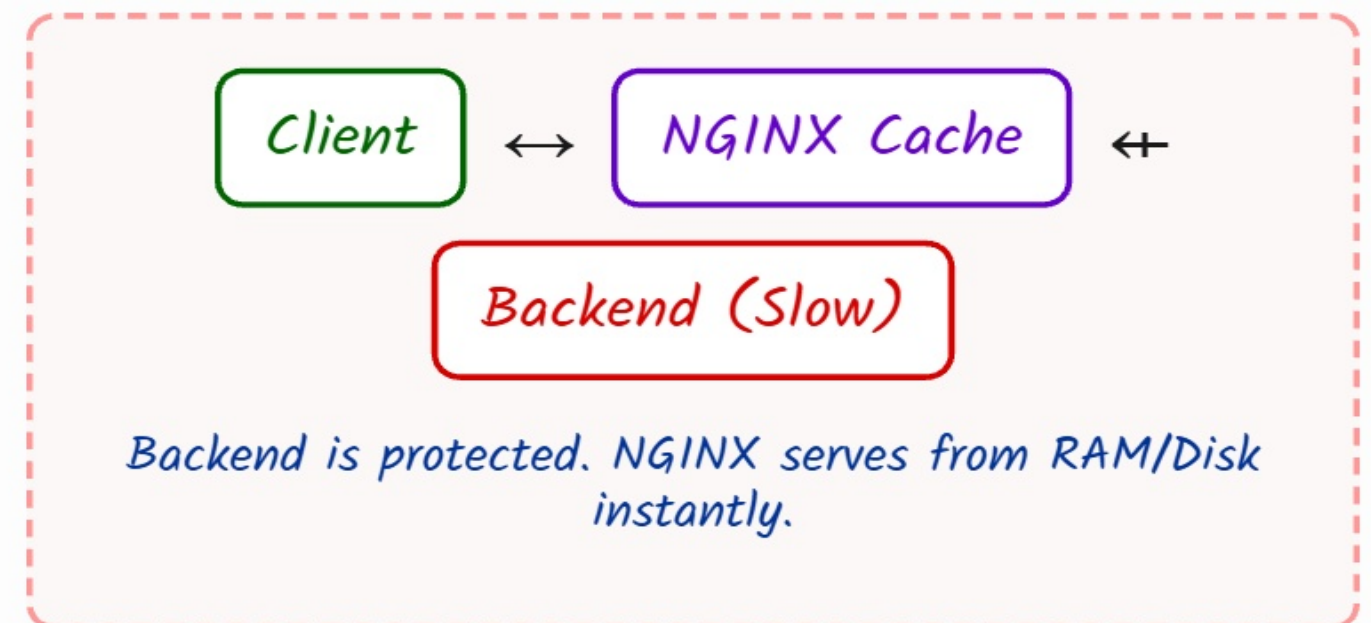
When NGINX proxies, the backend sees the connection coming from NGINX's IP (e.g., 127.0.0.1). We must forward the real client data.

```
location / {
    proxy_pass http://nodejs_backend;

    # Essential Proxy Headers
    proxy_set_header Host $host;
    proxy_set_header X-Real-IP
$remote_addr;
    proxy_set_header X-Forwarded-For
$proxy_add_x_forwarded_for;
    proxy_set_header X-Forwarded-Proto
$scheme;
}
```

## ★ 13. Caching & Buffering

NGINX can cache backend responses (Microcaching) to survive massive traffic spikes.



```
# In http {}
proxy_cache_path /var/cache/nginx
keys_zone=mycache:10m;

# In location {}
proxy_cache mycache;
proxy_cache_valid 200 302 10m;
proxy_cache_use_stale error timeout
updating;
```

⚠ proxy\_buffering on; (default) allows NGINX to absorb the backend response quickly, freeing up the backend, while NGINX slowly sends it to slow clients.

**Security Note:** Always set X-Forwarded-Proto so the backend knows if the original request was HTTPS, preventing infinite redirect loops!

## ★ 14. Upstream Context

The upstream block defines a pool of servers NGINX can route traffic to.

```
upstream backend_pool {  
    server 10.0.0.1:8080 weight=3;  
    server 10.0.0.2:8080;  
    server 10.0.0.3:8080 backup;  
}
```

Use `proxy_pass http://backend_pool;` in your location block to utilize this.

## ★ 15. Balancing Algorithms

- **Round Robin (Default):** 1-2-3-1-2-3. Distributed evenly.
- **Least Connections:** Sends to server with fewest active connections. Good for long-lived reqs.
- **IP Hash:** Uses client IP to hash to a specific server. Crucial for **Sticky Sessions** (stateful apps).
- **Hash:** Hash based on a custom variable like URI (`hash $request_uri;`). Good for cache locality.

## ★ 16. Passive Health Checks

Open Source NGINX monitors connections as they happen (Passive). If a server fails, it marks it down temporarily.



NGINX instantly tries Server 2.

Server 1 gets marked "down" for `fail_timeout`.

```
upstream backend {  
    server srv1.app max_fails=3  
    fail_timeout=30s;  
    server srv2.app max_fails=3  
    fail_timeout=30s;  
}
```

## ★ 17. Keepalive to Upstreams

By default, NGINX opens a *\*new\** TCP connection to the backend for every request. This wastes CPU/Ports!

⚠ Enable upstream Keepalives to maintain open connections to backends!

```
upstream backend {  
    server 10.0.0.1;  
    keepalive 32; # Idle connections to keep open  
}  
# Must also clear connection header in proxy:  
proxy_set_header connection "";  
proxy_http_version 1.1;
```

## ★ 18. Rate Limiting (Leaky Bucket)

Protects against DDoS and brute force. NGINX uses the **Leaky Bucket** algorithm.

### Leaky Bucket Algorithm

↓ Traffic Spikes

[ Bucket (Burst Queue) ]

↓ Constant Rate

Traffic pours in randomly, but leaks out at a strict defined rate (e.g., 10 req/sec). Overflow (Burst) is rejected (503).

```
# Define zone in http {} (10mb RAM, 5 req/sec)
```

```
limit_req_zone $binary_remote_addr  
zone=mylimit:10m rate=5r/s;
```

```
# Apply in location {}
```

```
limit_req zone=mylimit burst=10 nodelay;
```

**nodelay:** Processes the burst immediately without artificially delaying the response, but still enforces the cap.

## ★ 19. Core Performance Tuning

- **sendfile on;** Skips copying files to user-space buffer. Copies directly from disk to network socket (OS level). Huge speedup for static files.
- **tcp\_nopush on;** Sends HTTP response headers and file data in one single packet instead of multiple. (Used with sendfile).
- **tcp\_nodelay on;** Overrides Nagle's algorithm. Sends small packets immediately (crucial for WebSockets and Keep-Alive).

## ★ 20. Gzip & Security

Don't compress everything! Only text formats.

```
gzip on;  
gzip_comp_level 5; # 1-9. 5 is sweet spot.  
gzip_types text/plain text/css  
application/json;
```

### Hide NGINX Version:

```
server_tokens off; # Removes version from headers
```

### Security Headers:

```
add_header X-Frame-Options "SAMEORIGIN";  
add_header X-Content-Type-Options "nosniff";
```

**Final Thought:** NGINX is an incredibly powerful tool. Mastering the **Event Loop** concept, the **Location Matching order**, and the interaction between **Upstreams & Keepalives** is what separates a beginner from a senior admin.